UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida

Docente: Marlon Fuentes

HOJA DE TRABAJO 3

CUDA Y PARALELISMO DE DATOS

Suma vectorial, clasificación por umbral y comparación de rendimiento CPU/GPU

Estudiante Daniel Barillas
Carné 22193
Sección 40
Semestre indicado 2, 2025

Índice

1. Trazabilidad con el enunciado 1

2. Fundamentos del modelo CUDA 1
2.1. Flujo general . . . 2

3. Parte 1: suma de arreglos en GPU 2
3.1. Objetivo y datos . . . 2
3.2. Kernel . . . 2
3.3. Resultado observado . . . 3
3.4. Respuestas requeridas . . . 3
3.4.1. ¿Qué partes se ejecutan en CPU y cuáles en GPU? . . . 3
3.4.2. ¿Qué ocurre al cambiar grid y block? . . . 3
3.4.3. ¿Qué ocurre si no se reserva o libera memoria? . . . 4
3.4.4. ¿Qué ocurre si no se transfiere el resultado a CPU? . . . 4

4. Parte 2: clasificación por umbral 4
4.1. Relación con la plantilla . . . 4
4.2. Regla de clasificación . . . 4
4.3. Resultados . . . 5

5. Parte 3: versión CPU y comparación de tiempos 6
5.1. Versión exclusivamente CPU . . . 6
5.2. Diseño de la comparación justa . . . 6
5.3. Tabla preparada para la medición . . . 7

6. Validación integral 7

7. Reproducibilidad y limitaciones 7
7.1. Reproducibilidad . . . 8
7.2. Factores que afectan el tiempo . . . 8
7.3. Fecha del documento . . . 8

8. Conclusiones 8

A. Archivos principales 9

B. Comandos mínimos para completar la medición 9

Resumen ejecutivo

Esta hoja de trabajo desarrolla tres competencias fundamentales de CUDA. La primera parte implementa una suma de arreglos en GPU y analiza la separación entre host y device. La segunda completa la plantilla `vacio.cu` para clasificar valores respecto de un umbral. La tercera construye una versión secuencial en CPU y un comparador reproducible CPU/GPU que distingue el tiempo del kernel del costo completo de transferencias. Las salidas disponibles validan numéricamente las Partes 1 y 2 y la línea base CPU. El comparador queda listo para ejecutarse en el equipo CUDA de entrega, sin sustituir mediciones reales por estimaciones.

1. Trazabilidad con el enunciado

El enunciado oficial se conserva dentro del repositorio, junto con la plantilla original de la Parte 2. La Tabla 1 relaciona cada requisito con su evidencia.

Requisito	Implementación	Estado
Suma de A y B en GPU	`source/parte1_suma_cuda.cu`: memoria, copias, kernel e impresión	**Cumplido**
Respuestas de Parte 1	Sección 3.4 de este informe y README	**Cumplido**
Completar `vacio.cu`	`source/vacio.cu`, contrastado con la plantilla original	**Cumplido**
Versión exclusivamente CPU	`source/parte3_cpu_vs_gpu.cpp`	**Cumplido**
Medición CPU y GPU	`source/parte3_comparacion_cpu_gpu.cu`, con eventos CUDA y reloj CPU	**Pendiente de medición**
Informe en PDF	Fuente LaTeX, capturas integradas y salida en `output/pdf`	**Cumplido**
No incluir ejecutables	`build/` excluido mediante `.gitignore`	**Cumplido**

Cuadro 1: Trazabilidad de requisitos y entregables.

Dato pendiente y criterio de honestidad

El equipo disponible durante la preparación final no expone `nvcc` ni una GPU NVIDIA en WSL. Por esa razón, los tiempos GPU nuevos no se inventan ni se extrapolan desde otra máquina. El código, los comandos y la tabla están listos; los valores se incorporarán a partir de `results/parte3_comparacion_cpu_gpu.txt` cuando se ejecute en el entorno CUDA usado para la entrega.

2. Fundamentos del modelo CUDA

Un programa CUDA combina dos espacios de ejecución. El *host*, que corresponde a la CPU, coordina el programa y administra recursos. El *device*, que corresponde a la GPU, ejecuta kernels sobre una gran cantidad de hilos. En esta actividad se usa memoria separada: un puntero host no sustituye a un puntero device y los datos deben transferirse de forma explícita.

El índice global unidimensional se calcula mediante:

$$i = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$$

Cada hilo procesa un índice independiente. Como el número de hilos lanzados puede superar el número de datos, todos los kernels incluyen la condición `i <N`.

2.1. Flujo general

1. Crear e inicializar datos en el host.
2. Reservar memoria device con `cudaMalloc`.
3. Copiar entradas mediante `cudaMemcpyHostToDevice`.
4. Calcular grid y block.
5. Lanzar el kernel.
6. comprobar el lanzamiento y sincronizar.
7. Copiar resultados mediante `cudaMemcpyDeviceToHost`.
8. Validar y liberar recursos.

3. Parte 1: suma de arreglos en GPU

3.1. Objetivo y datos

La primera parte suma dos arreglos de $N = 16$ elementos. La inicialización es:

$$A[i] = i + 1, \qquad B[i] = 2i + 3$$

Por lo tanto, el valor esperado es:

$$C[i] = A[i] + B[i] = 3i + 4$$

3.2. Kernel

```
__global__ void sumarArreglos(
    const int *A, const int *B, int *C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}
```

El programa usa bloques de ocho hilos y dos bloques. La condición de límites mantiene el kernel correcto incluso si se cambia a una configuración que lance más hilos que elementos.

3.3. Resultado observado

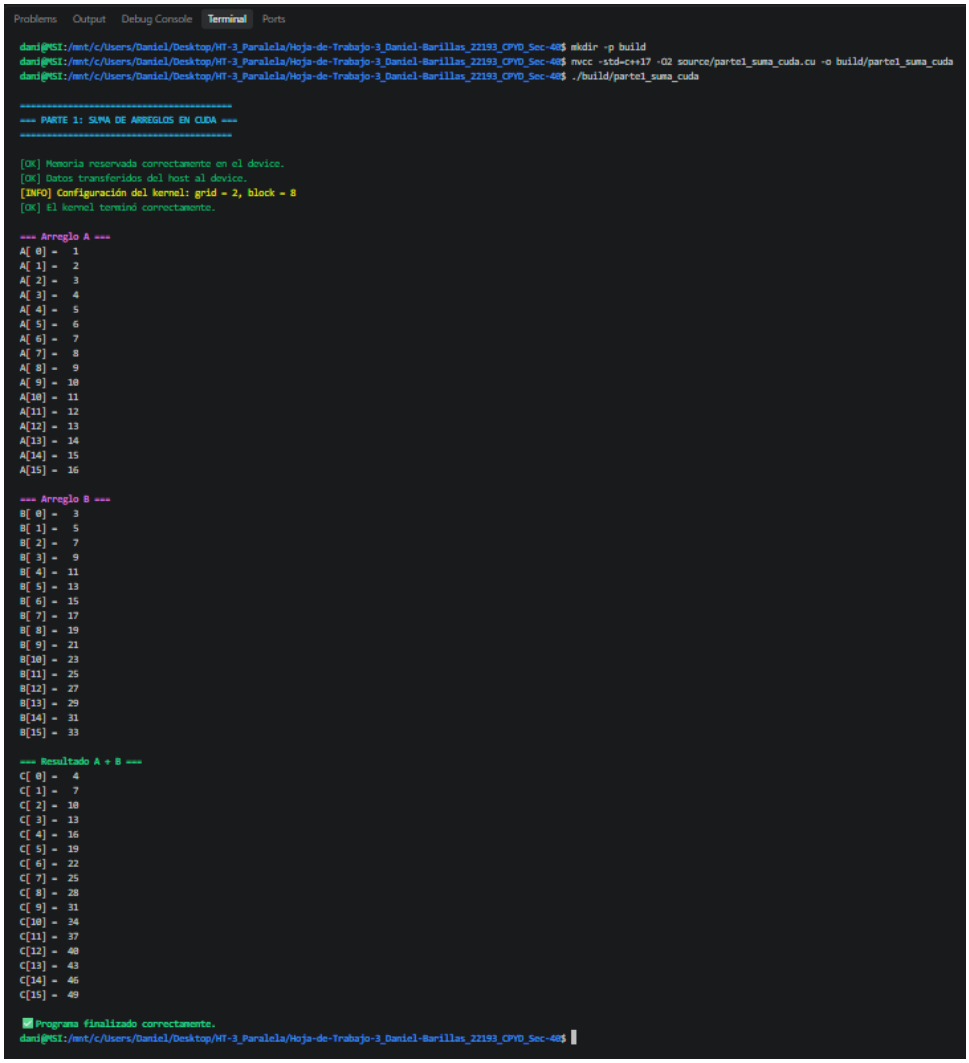


Figura 1: Compilación y ejecución de la suma de arreglos en CUDA.

La Figura 1 muestra $C[0] = 4$, $C[1] = 7$ y una progresión de tres unidades hasta $C[15] = 49$. Estos valores coinciden exactamente con $3i + 4$. La captura también evidencia la configuración `grid = 2` y `block = 8`, además del regreso de la salida al host.

3.4. Respuestas requeridas

3.4.1. ¿Qué partes se ejecutan en CPU y cuáles en GPU?

La CPU ejecuta `main`, inicializa A y B, invoca `cudaMalloc`, ordena las copias, calcula la configuración de lanzamiento, verifica errores, imprime la salida y libera memoria. La GPU ejecuta únicamente el cuerpo del kernel: cada hilo calcula un índice y suma dos enteros.

3.4.2. ¿Qué ocurre al cambiar grid y block?

El total de hilos es *gridSize* $\times$ *blockSize*. Si es menor que N, algunos elementos nunca se procesan. Si es mayor, la comprobación `idx < N` descarta los hilos sobrantes. El tamaño de bloque también afecta el rendimiento: bloques demasiado pequeños pueden desaprovechar recursos,

mientras bloques muy grandes pueden reducir ocupación o exceder límites del dispositivo. Por ello se usa la división de techo:

$$gridSize = \left\lceil \frac{N}{blockSize} \right\rceil$$

3.4.3. ¿Qué ocurre si no se reserva o libera memoria?

Sin `cudaMalloc`, las direcciones device no son válidas y las copias o el kernel devuelven errores. Liberar memoria antes de terminar causa accesos inválidos. No liberarla al final produce una fuga que puede agotar la memoria de la GPU en ejecuciones repetidas.

3.4.4. ¿Qué ocurre si no se transfiere el resultado a CPU?

El arreglo calculado permanece en `d_C`. `h_C` no se actualiza de forma automática porque host y device usan espacios de memoria distintos. Sin `cudaMemcpyDeviceToHost`, la CPU no dispone de los valores para imprimirlos o utilizarlos en otra operación.

4. Parte 2: clasificación por umbral

4.1. Relación con la plantilla

La actividad solicita completar `vacio.cu`. El repositorio conserva dos versiones:

- `docs/material_original/vacio_original.cu`: plantilla intacta;
- `source/vacio.cu`: solución completada y entregable.

También se mantiene `parte2_umbral_clasificacion.cu`, una variante con mensajes de diagnóstico, y `umbralizar.cu`, cuyo vocabulario sigue de cerca al archivo original.

4.2. Regla de clasificación

Para $N = 32$, el arreglo contiene $0, 5, 10, \ldots, 155$. La salida se define:

$$salida[i] = \begin{cases} 1, & entrada[i] \geq 50 \\ 0, & entrada[i] < 50 \end{cases}$$

```
__global__ void clasificarUmbral(
    const int *entrada, int *salida, int N, int umbral) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        salida[idx] = (entrada[idx] >= umbral) ? 1 : 0;
    }
}
```

4.3. Resultados

```
Problems  Output  Debug Console  Terminal  Ports

dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$ mkdir -p build
dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$ nvcc -std=c++17 -O2 source/parte2_umbral_clasificacion.cu -o build/parte2_umbral_clasificacion
dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$ ./build/parte2_umbral_clasificacion

========================================
=== PARTE 2: CATEGORIZACIÓN POR UMBRAL ===
========================================

[OK] Memoria reservada correctamente.
[OK] Datos transferidos al GPU.
[INFO] Configuración del kernel: grid = 1, block = 128
[OK] El kernel completó la clasificación.

=== Resultado de clasificación por umbral ===
Umbral = 50
in[ 0] =   0  -> out[ 0] = 0
in[ 1] =   5  -> out[ 1] = 0
in[ 2] =  10  -> out[ 2] = 0
in[ 3] =  15  -> out[ 3] = 0
in[ 4] =  20  -> out[ 4] = 0
in[ 5] =  25  -> out[ 5] = 0
in[ 6] =  30  -> out[ 6] = 0
in[ 7] =  35  -> out[ 7] = 0
in[ 8] =  40  -> out[ 8] = 0
in[ 9] =  45  -> out[ 9] = 0
in[10] =  50  -> out[10] = 1
in[11] =  55  -> out[11] = 1
in[12] =  60  -> out[12] = 1
in[13] =  65  -> out[13] = 1
in[14] =  70  -> out[14] = 1
in[15] =  75  -> out[15] = 1
in[16] =  80  -> out[16] = 1
in[17] =  85  -> out[17] = 1
in[18] =  90  -> out[18] = 1
in[19] =  95  -> out[19] = 1
in[20] = 100  -> out[20] = 1
in[21] = 105  -> out[21] = 1
in[22] = 110  -> out[22] = 1
in[23] = 115  -> out[23] = 1
in[24] = 120  -> out[24] = 1
in[25] = 125  -> out[25] = 1
in[26] = 130  -> out[26] = 1
in[27] = 135  -> out[27] = 1
in[28] = 140  -> out[28] = 1
in[29] = 145  -> out[29] = 1
in[30] = 150  -> out[30] = 1
in[31] = 155  -> out[31] = 1

✅ Finalizado (host).
dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$
```

Figura 2: Ejecución de la clasificación por umbral con mensajes de diagnóstico.

La Figura 2 confirma que los índices 0 a 9, cuyos valores son menores que 50, producen cero. El índice 10 contiene 50 y produce uno; todos los índices restantes mantienen esa clasificación.

```
Problems  Output  Debug Console  Terminal  Ports

dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$ nvcc -std=c++17 -O2 source/vacio.cu -o build/vacio
dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$ ./build/vacio
Umbral = 50
in[ 0] =   0  -> out[ 0] = 0
in[ 1] =   5  -> out[ 1] = 0
in[ 2] =  10  -> out[ 2] = 0
in[ 3] =  15  -> out[ 3] = 0
in[ 4] =  20  -> out[ 4] = 0
in[ 5] =  25  -> out[ 5] = 0
in[ 6] =  30  -> out[ 6] = 0
in[ 7] =  35  -> out[ 7] = 0
in[ 8] =  40  -> out[ 8] = 0
in[ 9] =  45  -> out[ 9] = 0
in[10] =  50  -> out[10] = 1
in[11] =  55  -> out[11] = 1
in[12] =  60  -> out[12] = 1
in[13] =  65  -> out[13] = 1
in[14] =  70  -> out[14] = 1
in[15] =  75  -> out[15] = 1
in[16] =  80  -> out[16] = 1
in[17] =  85  -> out[17] = 1
in[18] =  90  -> out[18] = 1
in[19] =  95  -> out[19] = 1
in[20] = 100  -> out[20] = 1
in[21] = 105  -> out[21] = 1
in[22] = 110  -> out[22] = 1
in[23] = 115  -> out[23] = 1
in[24] = 120  -> out[24] = 1
in[25] = 125  -> out[25] = 1
in[26] = 130  -> out[26] = 1
in[27] = 135  -> out[27] = 1
in[28] = 140  -> out[28] = 1
in[29] = 145  -> out[29] = 1
in[30] = 150  -> out[30] = 1
in[31] = 155  -> out[31] = 1
Finalizado (host).
dani@MSI:/mnt/c/Users/Daniel/Desktop/HT-3_Paralela/Hoja-de-Trabajo-3_Daniel-Barillas_22193_CPYD_Sec-40$
```

Figura 3: Ejecución de la plantilla `vacio.cu` después de completarla.

La Figura 3 valida específicamente el archivo solicitado por el enunciado. Ambas implementaciones producen la misma secuencia, lo que elimina la ambigüedad sobre cuál condición de umbral se usó.

5. Parte 3: versión CPU y comparación de tiempos

5.1. Versión exclusivamente CPU

El archivo `parte3_cpu_vs_gpu.cpp` implementa la misma suma mediante un ciclo secuencial. El vector de salida se calcula así:

```
void sumarArreglosCPU(
    const std::vector<int>& A,
    const std::vector<int>& B,
    std::vector<int>& C) {
    for (std::size_t i = 0; i < A.size(); ++i) {
        C[i] = A[i] + B[i];
    }
}
```

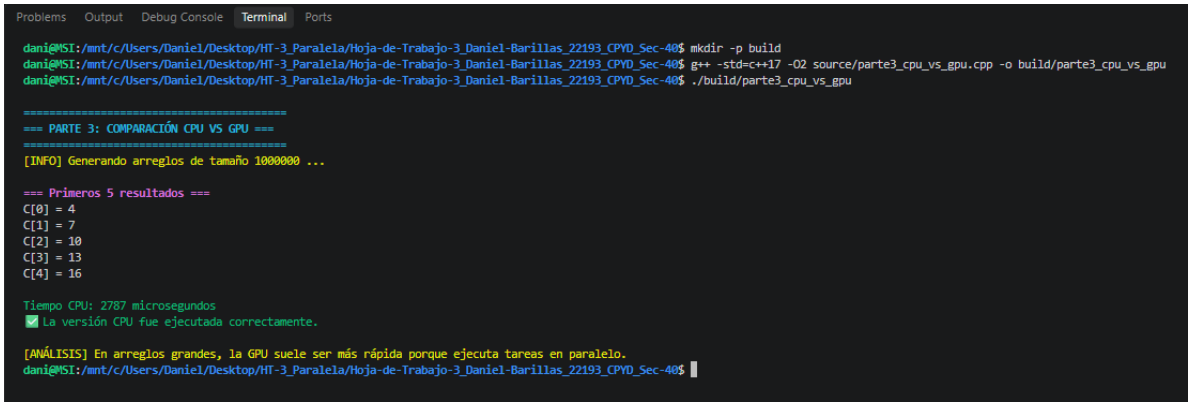


Figura 4: Evidencia histórica de la versión CPU: 2 787 microsegundos para un millón de elementos.

La Figura 4 valida la versión secuencial y sus primeros valores $4, 7, 10, 13, 16$. Ese tiempo es una línea base histórica y no se compara con la nueva versión GPU porque el tamaño del problema y la ejecución no son los mismos.

5.2. Diseño de la comparación justa

El nuevo archivo `parte3_comparacion_cpu_gpu.cu` ejecuta CPU y GPU sobre los mismos 8 388 608 elementos. Reserva los vectores de salida antes de medir, realiza un calentamiento CUDA y comprueba todos los resultados.

Se informan dos tiempos GPU:

1. **Solo kernel**: medido mediante `cudaEventRecord`; representa el cálculo en el dispositivo.

2. **Extremo a extremo**: incluye las copias H2D, el kernel y la copia D2H; excluye `cudaMalloc` y `cudaFree`.

Esta separación evita una conclusión engañosa. Una GPU puede ejecutar el kernel rápidamente y, aun así, perder ventaja si el problema es pequeño o las transferencias dominan el costo total.

5.3. Tabla preparada para la medición

Métrica	Resultado
Cantidad de elementos	8 388 608
Tiempo CPU	Pendiente de ejecución CUDA
Tiempo GPU, solo kernel	Pendiente de ejecución CUDA
Tiempo GPU, H2D + kernel + D2H	Pendiente de ejecución CUDA
Aceleración, solo kernel	Pendiente de ejecución CUDA
Aceleración extremo a extremo	Pendiente de ejecución CUDA
Discrepancias CPU/GPU	Debe ser 0

Cuadro 2: Campos que se completarán con la salida del comparador.

Comando para producir los datos

```
bash scripts/verificar_entorno.sh
bash scripts/compilar.sh
```

La salida relevante se guarda en: `results/parte3_comparacion_cpu_gpu.txt`.

6. Validación integral

Parte	Criterio	Resultado
Parte 1	$C[i] = 3i + 4$ para los 16 elementos	**Cumplido**
Parte 2	0 para valores menores que 50 y 1 desde 50	**Cumplido**
CPU	Primeros valores 4, 7, 10, 13 y 16; verificación completa	**Cumplido**
CPU/GPU	Mismos datos y cero discrepancias	**Pendiente de medición**
Memoria	Reserva, copia, sincronización y liberación comprobadas	**Cumplido**

Cuadro 3: Resumen de validación funcional.

La revisión del código añadió comprobaciones explícitas al retorno de `cudaDeviceSynchronize`. Esto evita anunciar que un kernel terminó correctamente cuando el lanzamiento fue válido, pero ocurrió un error durante su ejecución.

7. Reproducibilidad y limitaciones

7.1. Reproducibilidad

El script `scripts/compilar.sh` compila y ejecuta todos los programas. Cada salida se conserva como texto en `results/`. Los ejecutables se guardan en `build/`, directorio excluido de la entrega para respetar la instrucción de no entregar binarios.

7.2. Factores que afectan el tiempo

Los tiempos dependen del procesador, modelo de GPU, versión del controlador, frecuencias, temperatura, procesos concurrentes y optimizaciones del compilador. Por ello:

- no se comparan capturas obtenidas con tamaños diferentes;
- no se trasladan tiempos desde otro equipo;
- se documenta qué operaciones incluye cada medición;
- se valida corrección antes de interpretar velocidad.

7.3. Fecha del documento

El enunciado indica simultáneamente “Semestre 2, 2025” y “miércoles 21 de octubre”. El informe conserva el semestre declarado y evita generar una fecha automática con `\textbackslash today`, que podría introducir un año ajeno a la actividad.

8. Conclusiones

La suma vectorial demuestra un caso de paralelismo de datos directo: cada hilo trabaja sobre un elemento y no necesita comunicarse con otros hilos. Los resultados $4, 7, 10, \ldots, 49$ confirman que la indexación, las transferencias y el kernel son correctos.

La clasificación por umbral consolida el mismo patrón con una decisión independiente por dato. La transición exacta en 50 valida la condición `entrada[idx] >= umbral` y demuestra que la plantilla original fue completada de acuerdo con el enunciado.

La Parte 3 distingue correctamente entre demostrar paralelismo y demostrar rendimiento. No basta afirmar que una GPU es más rápida: deben usarse los mismos datos, establecer los límites temporales y comprobar la salida. El comparador implementado mide por separado el kernel y el ciclo completo, permitiendo analizar tanto capacidad de cálculo como costo de comunicación.

En conjunto, la actividad evidencia el ciclo completo de una aplicación CUDA: preparación en CPU, administración explícita de memoria, distribución en hilos, sincronización, retorno de datos y validación. La única evidencia pendiente es la medición específica de hardware, que debe obtenerse en el equipo CUDA y colocarse en la Tabla 2.

A. Archivos principales

Archivo	Propósito
source/parte1_suma_cuda.cu	Suma paralela y respuestas de Parte 1
source/vacio.cu	Plantilla de Parte 2 completada
source/parte2_umbral_clasificacion.cu	Variante documentada del umbral
source/parte3_cpu_vs_gpu.cpp	Versión exclusivamente CPU
source/parte3_comparacion_cpu_gpu.cu	Benchmark CPU/GPU justo
scripts/verificar_entorno.sh	Verificación de compiladores y GPU
scripts/compilar.sh	Compilación, ejecución y captura de resultados
docs/ENUNCIADO.md	Transcripción del enunciado
docs/VALIDACION.md	Procedimiento de reproducción

B. Comandos mínimos para completar la medición

```
cd /ruta/al/repositorio
bash scripts/verificar_entorno.sh
bash scripts/compilar.sh
cat results/parte3_comparacion_cpu_gpu.txt
```